Assignment 2 Report

Functions, Classes & Object-Oriented Programming (CO2)

Comprehensive Design, Implementation, Evaluation & Day-4 Challenge

---

Student Name: NEELAKANDA MOORTHY

Register Number: 192421063

Course: CO2 - Object-Oriented Programming in C++

Bloom's Taxonomy Level: K5 - Evaluate (All Tasks)

Total Marks: 50 Marks

Compiler & Version: MinGW-W64 GCC 8.1.0 (C++17)

Compilation Flags: g++ -std=c++17 -Wall -Wextra

GitHub Repository: github.com/moorthy-dnkkw/DSA0119-C-

Table of Contents & Executive Summary

This report documents the design, verification, empirical performance evaluation, and Day-4 design challenge for Assignment 2 (CO2). All tasks are assessed at Bloom's Level K5 (Evaluate).

Task	Title	Bloom Level	Marks	Core Concept Evaluated
Q1 / Task 01	Value, Reference & Pointer	K5 - Evaluate	6	Parameter passing mechanics & storage
Q2 / Task 02	Return by Reference Dangers	K5 - Evaluate	4	Lvalue references & dangling refs
Q3 / Task 03	Default Args & Overloading	K5 - Evaluate	6	Ambiguity & overload resolution
Q4 / Task 04	Class with Internal Array	K5 - Evaluate	6	Scope resolution & shift vs ring buffer
Q5 / Task 05	Static Members & Array of Obj	K5 - Evaluate	7	Class-level state & static persistence
Q6 / Task 06	Friend & Inline Functions	K5 - Evaluate	8	Symmetric ops & 10M call benchmark
Q7 / Task 07	Day-4 Design Challenge	K5 - Evaluate	13	Diagnose O(K) shift, O(1) ring redesign

Q1 (Task 01) - Call by Value, by Reference and by Pointer (6 Marks)

Problem Context: Hostel Fee Concession Updater

A student hostel management system applies concessions (Merit 10%, Sports 15%). When passing by value, calculations inside the function do not affect the stored fee of the student. The challenge requires implementing by value, by reference, and by pointer, predicting outputs, and defending the interface design.

Predict the Output Analysis

```
Given Program:
  void change(int a, int& b, int* c) { a += 10; b += 10; *c += 10; }
  int& bigger(int& p, int& q) { return p > q ? p : q; }
  void show(int x, int y = 5, int z = 10) { std::cout << x + y + z << ' '; }

Predicted Output (Written Before Running):
  1 12 13
  1 100
  16 13 6

Actual Output (Exact Match):
  1 12 13
  1 100
  16 13 6
```

- change(a, b, &c): a is copied (remains 1); b is aliased (becomes 12); c is dereferenced (becomes 13).
- bigger(a, c) = 100: c > a (13 > 1) -> returns reference to c; assigning 100 mutates c directly to 100.
- show(): show(1)=1+5+10=16; show(1,2)=1+2+10=13; show(1,2,3)=1+2+3=6.

Complete the Code: Hostel Fee Concession

```
void applyConcession(long long& fee, int percent) {
    fee -= fee * percent / 100; // Call by reference modifies caller fee
}
Output:
  after call by value : 45000
  after call by reference: 40500
```

Parameter Passing Mechanisms & Storage

Passing Style	Storage Location	Caller Mutation Capability & Rules
Call by Value	Callee stack activation record	No mutation; local copy destroyed upon function return
Call by Reference	Machine register / stack address	Directly mutates caller variable; syntactic alias
Call by Pointer	Explicit 8-byte pointer on stack	Mutates caller variable via explicit pointer dereference (*ptr)
Const Reference	Machine register / stack address	Read-only access; avoids expensive copy of large structures

Design Decision & Alternative Concept Analysis

Decision: Return the new fee (functional style). It ensures immutability, auditability, thread-safety, and chaining. For primitives (long long), call-by-value copies only 8 bytes. For a 256-byte Student object, const Student& passes an 8-byte address, achieving a 32x memory copy savings.

Q2 (Task 02) - Return by Reference and Its Dangers (4 Marks)

Return by Reference: Examination Moderation

Returning an lvalue reference allows using the function call on the left-hand side of an assignment operator (=):

```
int& highestMark(int marks[], int size) { ... return marks[maxIdx]; }
// Moderation applied directly on LHS:
highestMark(marks, n) += 2; // Directly updates marks[3] from 91 to 93!
```

Compiler Warning on Local Reference & Undefined Behaviour

```
int& dangerousLocal() {
    int local = 42;
    return local; // WARNING: reference to local variable returned
}
Compiler Diagnostic (MinGW-W64 GCC 8.1.0):
  warning: reference to local variable 'local' returned [-Wreturn-local-addr]
```

Why it is Undefined Behaviour: The variable local is allocated on dangerousLocal's activation frame. Upon return, the stack frame is popped. The returned reference points to deallocated memory. Accessing it reads corrupted garbage or triggers segmentation faults.

Changed Condition: std::vector Growth Invalidating References

When std::vector exceeds capacity, it reallocates a larger memory buffer elsewhere and moves all elements. Any reference obtained prior to reallocation becomes a dangling reference pointing to freed memory.

```
Empirical Demonstration:
  Address of v[0] before push_back: 0xe86150
  Address of v[0] after reallocation: 0xe867d0 (Addresses differ -> Dangling!)
  Safe Solution: Return array/vector index (int) instead of reference.
```

Q3 (Task 03) - Default Arguments and Overloading (6 Marks)

Ambiguity Problem Diagnosis

The original declarations caused ambiguity:

```
long long charge(double w);                                          // (1)
long long charge(double w, bool express);                            // (2)
long long charge(double w, bool express = false, bool insured = false);  // (3)
Error: charge(2.5) matches (1) and matches (3) with default args -> Ambiguous!
```

Redesigned Overloaded Functions & 6 Prediction Tests

Call Invocation	Predicted Overload	Conversion Type	Calculated Fee
charge(500)	charge(int)	Exact Match (int literal)	25000 paise (Rs 250.00)
charge(2.5)	charge(double)	Exact Match (double literal)	125000 paise (Rs 1250.00)
charge(2.0f)	charge(double)	Promotion (float -> double)	100000 paise (Rs 1000.00)
charge(2)	charge(int)	Exact Match (int beats conv)	100 paise (Rs 1.00)
charge(p, true, false)	charge(const Parcel&)	Exact Match (Object reference)	112500 paise (Rs 1125.00)
charge(500, true)	charge(int, bool, bool)	Exact Match with 1 default	37500 paise (Rs 375.00)

Overload Priority & Right-to-Left Defaults Rule

- Overload Priority: Exact Match > Promotion (float->double) > Standard Conversion > User-defined Conversion.
- Right-to-Left Rule: Arguments match left-to-right. Default arguments must be contiguous on rightmost parameters because C++ has

Q4 (Task 04) - Class with an Array Inside It (6 Marks)

ATM Mini-Statement Class Specification

The SavingsAccount class encapsulates account number, balance in integer paise, and an internal array of the last 10 transactions. Functions deposit(), withdraw(), and miniStatement() are defined outside using the scope resolution operator ::.

Handling the 11th Transaction: Shift O(K) vs Circular Buffer O(1)

```
Shift Approach (O(K)): Moves elements 1..9 to 0..8 via memmove, writes at index 9.
Circular Buffer (O(1)): Overwrites oldest slot at head and advances head = (head + 1) % K.

Output Mini-Statement (After 12 transactions - both show identical last 10):
  1. DR Rs 200.00   2. CR Rs 300.00   3. DR Rs 100.00   4. CR Rs 750.00
  5. DR Rs 50.00    6. CR Rs 1200.00  7. DR Rs 600.00   8. CR Rs 400.00
  9. CR Rs 250.00  10. DR Rs 350.00   Balance: Rs 13600.00
```

Design Decision: Last 50 Transactions

For K = 50, circular buffer requires zero code restructuring (only array size constant changes) and keeps O(1) performance. The shifting approach must copy 49 elements per transaction, degrading throughput.

Q5 (Task 05) - Static Members and Arrays of Objects (7 Marks)

Predict the Output: Token Class

```
Token counter[3]; Token extra;
std::cout << counter[0].get() << ' ' << counter[2].get() << ' '
          << extra.get() << ' ' << Token::issued() << '\n';

Predicted Output: 101 103 104 4
Actual Output   : 101 103 104 4 (Exact Match)
```

Complete the Code: BusTicket with Static Capacity Tracking

```
class BusTicket {
    static int issued; static const int CAPACITY = 3;
    int seat; std::string name;
public:
    explicit BusTicket(const std::string& n) : seat(0), name(n) {
        if (issued < CAPACITY) { ++issued; seat = issued; } // TODO 1
    }
    static int count() { return issued; }                   // TODO 2
};
Expected & Actual Output:
  Asha: Seat 1
  Bala: Seat 2
  Chitra: Seat 3
  Dev: Waiting
  Issued: 3
```

Static Rules & Lifetime Persistence across Calls

- Static members belong to the class, not instances. Defined outside class in static storage.
- Static member functions have no this pointer; cannot access non-static members directly.
- When tickets are created in a function called twice, static count persists across calls.

Q6 (Task 06) - Friend Functions and Inline Functions (8 Marks)

Friend Functions Implementation

```
inline int minutesBetween(const Time& a, const Time& b) {
    int totalA = a.h * 60 + a.m; int totalB = b.h * 60 + b.m;
    return (totalA > totalB) ? (totalA - totalB) : (totalB - totalA); // TODO 1
}
inline bool earlier(const Time& a, const Time& b) {
    return (a.h < b.h) || (a.h == b.h && a.m < b.m);                // TODO 2
}

Execution Verification:
  Time departure(9, 45), arrival(13, 10);
  Output: 205 false (Expected: 205 false - EXACT MATCH)
```

Symmetric Operations & Inline Optimization

- Why Friend? minutesBetween(a, b) treats both operands symmetrically without favoring either.
- Inline Hint: Asks compiler to substitute function body at call site, eliminating stack push/pop.
- 10,000,000 Call Growth Benchmark: Executed in 15.52 ms under GCC 8.1.0.

Q7 (Task 07) - Day-4 Design Challenge (13 Marks)

A. Diagnose the Limitation (2 Marks)

The original SavingsAccount ledger shifts array elements via memmove on every transaction once full. For N = 1,000,000 transactions and K = 100 history slots, it performs ~99 million element memory copies, causing CPU cache pollution and write-stall latency spikes.

B. Define the Design Objective (2 Marks)

- Reduce insertion time complexity from O(K) to strictly O(1) constant time.
- Eliminate 100% of internal element memory copies.
- Achieve >2.0x execution speedup while retaining 100% chronological output fidelity.

C. Design the Revised Solution (4 Marks)

Redesigned as CircularLedgerAccount<K> ring buffer. Uses head pointer and modular indexing ((head + count) % K). Inserts in O(1) time with 0 element copies. Memory overhead is only 8 bytes (head, count).

D. Predict Before Implementing (1 Mark)

Predicted: For N = 1,000,000 and K = 100, circular ledger will run ~2.5x to 3.5x faster than shifting ledger and produce identical mini-statement outputs.

E. Implement and Validate the Redesign (2 Marks)

Metric / Feature	Original Shifting Ledger	Redesigned Circular Ring	Performance Advantage
Insertion Complexity	O(K) per insert	O(1) strictly constant	Redesign [Strict O(1)]
Element Copies (1M txns)	~99 Million copies	0 copies	Redesign [Zero Copies]
Execution Time (1M txns)	7.997 ms	3.008 ms	Redesign [2.66x FASTER!]
Auxiliary Memory	0 bytes	8 bytes (head, count)	Original (+8 bytes negligible)
Heap Allocation	None (Zero Heap)	None (Zero Heap)	Tie (Cache Contiguous)
Output Match	Verified	Verified (Exact Match)	100% Behavioral Parity

F. Defend the Final Design (2 Marks)

The circular ring buffer ledger achieved all design objectives: O(1) guaranteed latency, zero memory copies, 2.66x speedup, and 100% output accuracy. For any production banking system with K >= 10, the circular buffer is unequivocally superior.